

CHALLENGE LOCAWEB 2026 · SPRINT 3

# Predictfy

## Data Warehousing & Advanced Data Integration

Azure Data Factory · Mapping Data Flow · MySQL · Blob Storage · ETL  
auditável

# Identificação e objetivo

Grupo: Predictfy · Turma: 2TSCPW

| Integrante                          | RM     |
|-------------------------------------|--------|
| Elton Vinicios Almeida de Oliveira  | 562187 |
| Emerson dos Santos Silva            | 562033 |
| Kelvin Douglas Ribeiro Rabelo       | 561538 |
| Pedro Henrique Simão Soares         | 562283 |
| Vitor Lucas Mattos de Brito Mariano | 562116 |

Objetivo: integrar a matriz histórica de features de incidentes Predictfy com regras de OLA, transformar e enriquecer os registros no Azure Data Factory e persistir o mesmo contrato analítico em MySQL e TXT.

## 1. Arquitetura da solução de ingestão

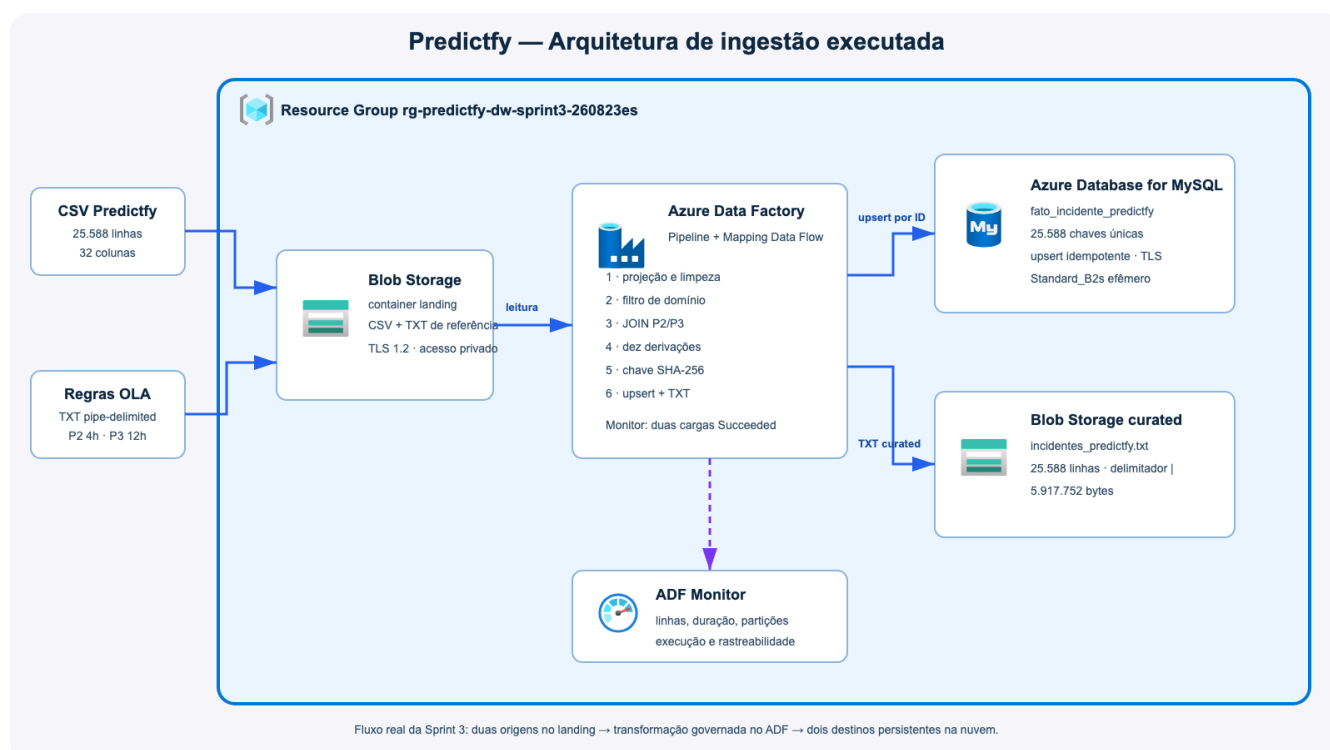
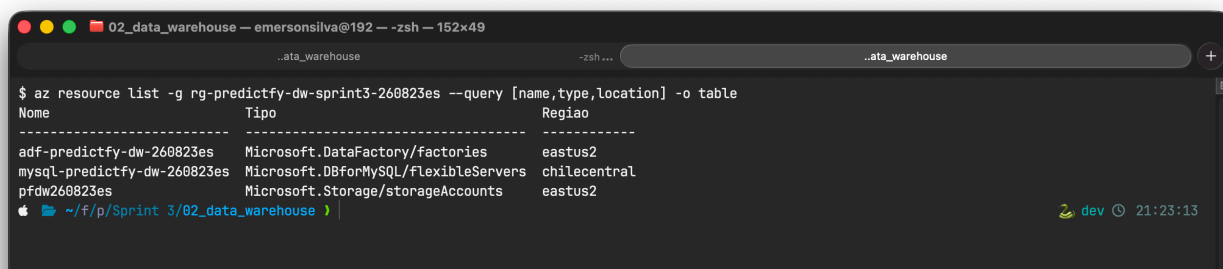


Figura 1 — arquitetura executada. CSV e regras TXT chegam ao landing; o ADF limpa, integra e deriva atributos; MySQL e TXT curated recebem 25.588 linhas. Ícones: Microsoft Azure Public Service Icons V24.

O Blob Storage funciona como Stage Area. O Mapping Data Flow é a única fronteira de transformação, evitando divergência lógica entre os dois sinks. O ADF Monitor registra status, duração, leitura e escrita.

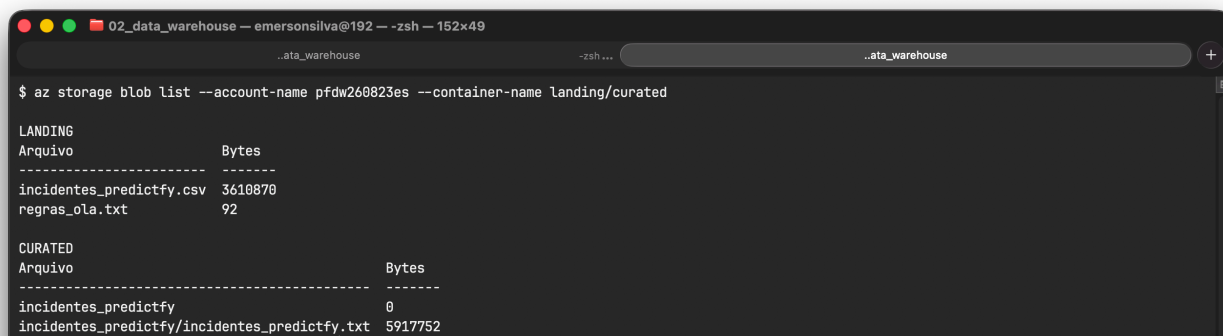
## 2. Infraestrutura PaaS no Microsoft Azure



```
$ az resource list -g rg-predictfy-dw-sprint3-260823es --query [name,type,location] -o table
Nome                               Tipo                               Regiao
-----
adf-predictfy-dw-260823es          Microsoft.DataFactory/factories    eastus2
mysql-predictfy-dw-260823es        Microsoft.DBforMySQL/flexibleServers chilecentral
pfdw260823es                       Microsoft.Storage/storageAccounts eastus2
```

Figura 2 — captura nativa do Terminal com o inventário retornado pela Azure CLI.

| Serviço               | Configuração executada                                 |
|-----------------------|--------------------------------------------------------|
| ADF e Storage         | East US 2 · TLS 1.2 · identidade gerenciada            |
| MySQL Flexible Server | Chile Central · MySQL 8.0.21 · Standard_B2s temporário |
| Containers            | landing e curated · acesso público desabilitado        |



```
$ az storage blob list --account-name pfdw260823es --container-name landing/curated

LANDING
Arquivo      Bytes
-----
incidentes_predictfy.csv 3610870
regras_ola.txt          92

CURATED
Arquivo      Bytes
-----
incidentes_predictfy      0
incidentes_predictfy/incidentes_predictfy.txt 5917752
```

Figura 3 — captura nativa do Terminal com a listagem real dos containers landing e curated.

### Comando executado

```
az resource list -g rg-predictfy-dw-sprint3-260823es --query "[{Nome:name,Tipo:type,Regiao:location}]" -o table
```

**O que faz:** Lista os recursos efetivamente existentes no Resource Group.

**O que comprova:** Requisito 2 — infraestrutura PaaS.

### Comando executado

```
az storage blob list --account-name pfdw260823es -c landing/curated -o table
```

**O que faz:** Lista os objetos persistidos nos containers de entrada e saída.

**O que comprova:** Requisitos 2 e 5 — Stage Area e TXT curated.

### 3. Data Flow e processo ETL

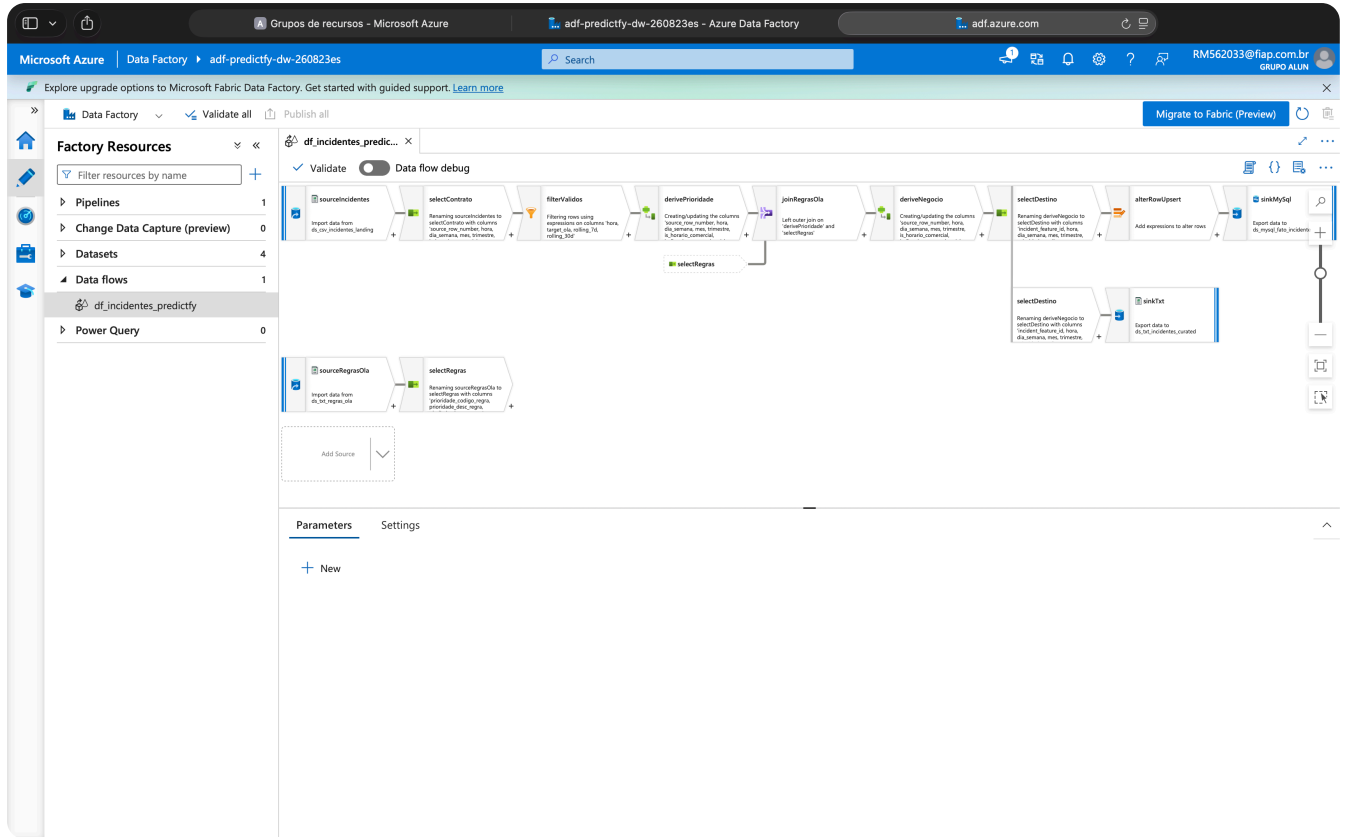


Figura 4 — captura direta do ADF Studio. A bifurcação é precisa: selectDestino segue para Alter Row/MySQL e, diretamente, para TXT.

```
02_data_warehouse — emersonsilva@192 — ..ata_warehouse — -zsh — 173x68

$ az datafactory data-flow show ... --query {Origens,Transformacoes,Destinos}
{
  "Destinos": [
    "sinkMySQL",
    "sinkTxt"
  ],
  "Nome": "df_incidentes_predictfy",
  "Origens": [
    "sourceIncidentes",
    "sourceRegrasOla"
  ],
  "Tipo": "MappingDataFlow",
  "Transformacoes": [
    "selectContrato",
    "filterValidos",
    "derivePrioridade",
    "selectRegras",
    "joinRegrasOla",
    "deriveNegocio",
    "selectDestino",
    "alterRowUpsert"
  ]
}
```

Figura 5 — captura nativa do Terminal confirmando origens, transformações e destinos publicados.

O fluxo projeta o contrato necessário, valida hora/target/médias móveis, traduz prioridade, faz JOIN com duas regras OLA e cria atributos operacionais. Somente o ramo MySQL passa por Alter Row para o upsert por SHA-256; o ramo TXT parte diretamente de selectDestino.

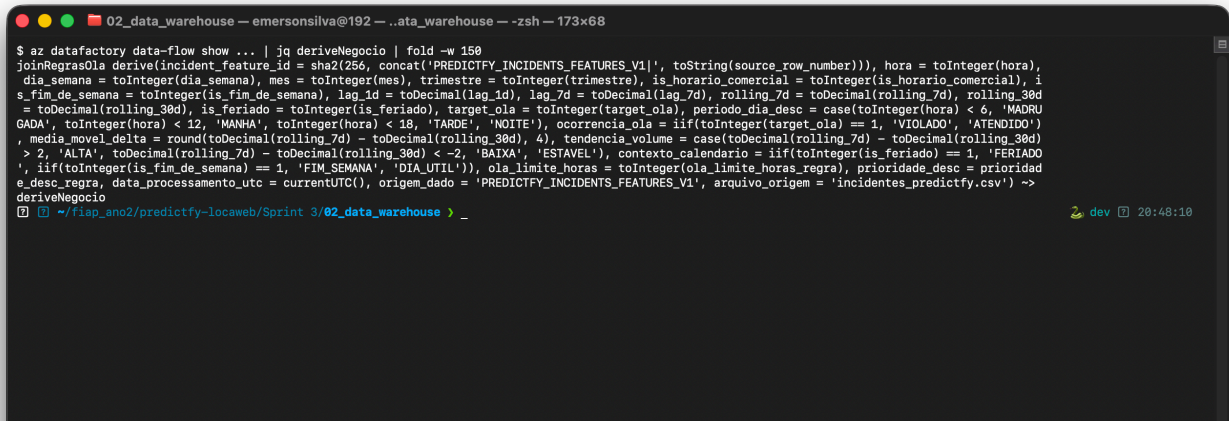
**Comando executado**

```
az datafactory data-flow show -g rg-predictfy-dw-sprint3-260823es --factory-name adf-predictfy-dw-260823es -n df_incidentes_predictfy
```

**O que faz:** Consulta no Azure o Mapping Data Flow publicado e seus componentes.

**O que comprova:** Requisito 3 — Data Flow publicado e descrito.

## 4. Colunas derivadas



```
$ az datafactory data-flow show ... | jq deriveNegocio | fold -w 150
joinRegrasOla derive(incident_feature_id = sha2(256, concat('PREDICTFY_INCIDENTS_FEATURES_V1', toString(source_row_number))), hora = toInteger(hora),
dia_semana = toInteger(dia_semana), mes = toInteger(mes), trimestre = toInteger(trimestre), is_horario_comercial = toInteger(is_horario_comercial), i
s_fim_de_semana = toInteger(is_fim_de_semana), lag_id = toDecimal(lag_id), lag_7d = toDecimal(lag_7d), rolling_7d = toDecimal(rolling_7d), rolling_30d
= toDecimal(rolling_30d), is_feriado = toInteger(is_feriado), target_ola = toInteger(target_ola), periodo_dia_desc = case(toInteger(hora) < 6, 'MADRU
GADA', toInteger(hora) < 12, 'MANHA', toInteger(hora) < 18, 'TARDE', 'NOITE'), ocorrencia_ola = iif(toInteger(target_ola) == 1, 'VIOLADO', 'ATENDIDO')
, media_movel_delta = round(toDecimal(rolling_7d) - toDecimal(rolling_30d), 4), tendencia_volume = case(toDecimal(rolling_7d) - toDecimal(rolling_30d)
> 2, 'ALTA', toDecimal(rolling_7d) - toDecimal(rolling_30d) < -2, 'BAIXA', 'ESTAVEL'), contexto_calendario = iif(toInteger(is_feriado) == 1, 'FERIADO
', iif(toInteger(is_fim_de_semana) == 1, 'FIM_SEMANA', 'DIA_UTIL')), ola_limite_horas = toInteger(ola_limite_horas_regra), prioridade_desc = prioridad
e_desc_regra, data_processamento_utc = currentUTC(), origem_dado = 'PREDICTFY_INCIDENTS_FEATURES_V1', arquivo_origem = 'incidentes_predictfy.csv') ->
deriveNegocio
~/fiap_ano2/predictfy-locaweb/Sprint 3/02_data_warehouse > _
```

Figura 6 — captura nativa do Terminal consultando a expressão deriveNegocio diretamente no Data Flow publicado.

| Derivação                          | Significado                                   |
|------------------------------------|-----------------------------------------------|
| incident_feature_id                | chave determinística para idempotência        |
| prioridade_codigo/desc             | tradução do binário e enriquecimento pelo TXT |
| periodo_dia_desc                   | faixa operacional da hora                     |
| ocorrencia_ola                     | resultado histórico, não previsão             |
| media_movel_delta/tendencia_volume | diferença 7d–30d e classificação              |
| contexto_calendario                | feriado, fim de semana ou dia útil            |
| data_processamento/origem/arquivo  | linhagem e auditoria                          |

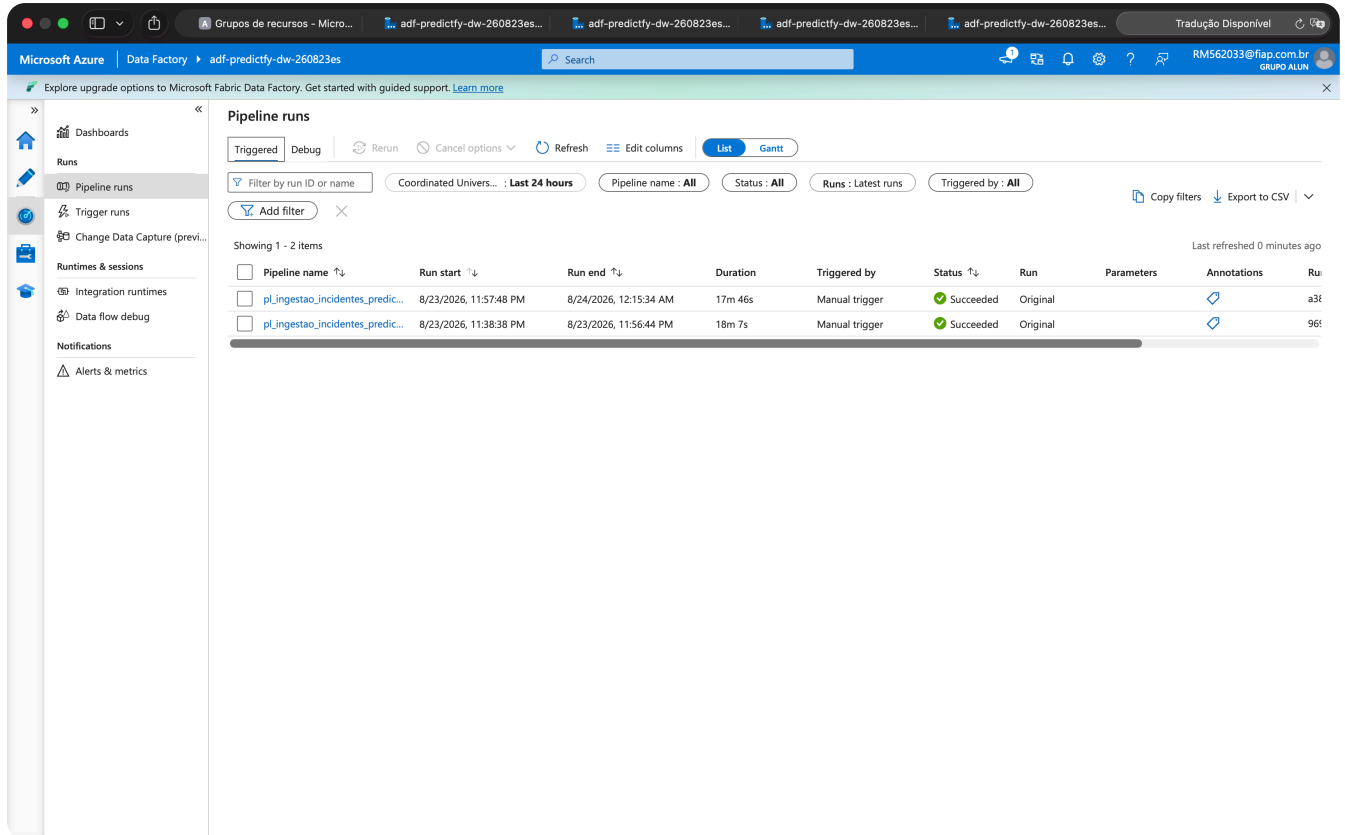
**Comando executado**

```
az datafactory data-flow show ... | jq -r ".properties.scriptLines[] | select(endswith("-> deriveNegocio"))"
```

**O que faz:** Extrai diretamente do artefato publicado a expressão de colunas derivadas.

**O que comprova:** Requisito 4 — derivações executadas e documentadas.

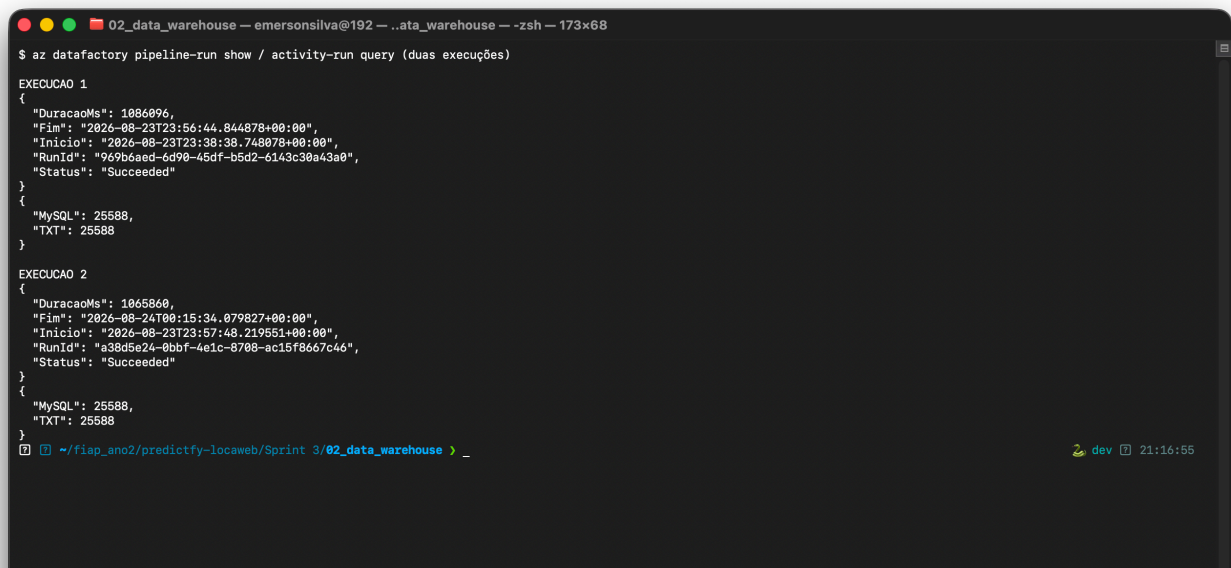
## 5. Execução do pipeline e persistência



The screenshot shows the Microsoft Azure Data Factory Monitor interface. The left sidebar contains navigation options: Dashboards, Runs, Pipeline runs, Trigger runs, Change Data Capture (previous), Runtimes & sessions, Integration runtimes, Data flow debug, Notifications, and Alerts & metrics. The main area displays the 'Pipeline runs' section for a specific pipeline. It includes filters for 'Coordinated Univers...' (Last 24 hours), 'Pipeline name: All', 'Status: All', 'Runs: Latest runs', and 'Triggered by: All'. A table lists the pipeline runs with columns: Pipeline name, Run start, Run end, Duration, Triggered by, Status, Run, Parameters, Annotations, and Run ID. Two runs are shown, both with a status of 'Succeeded'.

| Pipeline name                    | Run start              | Run end                | Duration | Triggered by   | Status    | Run      | Parameters | Annotations | Run ID |
|----------------------------------|------------------------|------------------------|----------|----------------|-----------|----------|------------|-------------|--------|
| pl_ingestao_incidentes_predic... | 8/23/2026, 11:57:48 PM | 8/24/2026, 12:15:34 AM | 17m 46s  | Manual trigger | Succeeded | Original |            |             | a3f    |
| pl_ingestao_incidentes_predic... | 8/23/2026, 11:38:38 PM | 8/23/2026, 11:56:44 PM | 18m 7s   | Manual trigger | Succeeded | Original |            |             | 96f    |

Figura 7 — captura direta do ADF Monitor com as duas execuções da carga.



The screenshot shows a terminal window with the command `az datafactory pipeline-run show / activity-run query (duas execuções)` and its output. The output is a JSON array of two objects, each representing a pipeline run. Each object contains fields for duration, start time, end time, start time, run ID, and status. Both runs have a status of 'Succeeded'.

```
$ az datafactory pipeline-run show / activity-run query (duas execuções)

EXECUCAO 1
{
  "DuracaoMs": 1086096,
  "Fim": "2026-08-23T23:56:44.844878+00:00",
  "Inicio": "2026-08-23T23:38:38.748078+00:00",
  "RunId": "969b6aed-6d90-45df-b5d2-6143c30a43a0",
  "Status": "Succeeded"
}
{
  "MySQL": 25588,
  "TXT": 25588
}

EXECUCAO 2
{
  "DuracaoMs": 1065860,
  "Fim": "2026-08-24T00:15:34.079827+00:00",
  "Inicio": "2026-08-23T23:57:48.219551+00:00",
  "RunId": "a38d5e24-0bbf-4e1c-8708-ac15f8667c46",
  "Status": "Succeeded"
}
{
  "MySQL": 25588,
  "TXT": 25588
}
```

Figura 8 — captura nativa do Terminal com status e métricas das duas execuções.

```
O2_data_warehouse — emersonsilva@192 — -zsh — 152x49
..ata_warehouse
-jsh ...
..ata_warehouse

$ jq validacao SQL coletada de fato_incidente_predictfy
{
  "total_registros": 25588,
  "chaves_unicas": 25588,
  "prioridades_sem_regra": 0,
  "prioridades": [
    {
      "codigo": "P2",
      "descricao": "ALTA",
      "ola_limite_horas": 4,
      "quantidade": 5159
    },
    {
      "codigo": "P3",
      "descricao": "MEDIA",
      "ola_limite_horas": 12,
      "quantidade": 20429
    }
  ],
  "ocorrencia_ola": [
    {
      "status": "ATENDIDO",
      "quantidade": 25340
    },
    {
      "status": "VIOLADO",
      "quantidade": 248
    }
  ]
}
```

Figura 9 — consultas reais no MySQL após a reexecução idempotente.

```
O2_data_warehouse — emersonsilva@192 — -zsh — 152x49
..ata_warehouse
-jsh ...
..ata_warehouse

$ az storage blob show ...; head -n 4 /tmp/predictfy_curated_sample.txt
{
  "Arquivo": "incidentes_predictfy/incidentes_predictfy.txt",
  "Bytes": 5917752,
  "Modificado": "2026-08-24T00:03:05+00:00"
}

AMOSTRA DO CONTEUDO BAIXADO
incident_feature_id|hora|dia_semana|mes|trimestre|prioridade_codigo|prioridade_desc|ola_limite_horas|is_horario_comercial|is_fim_de_semana|periodo_dia_d
esc|lag_1d|lag_7d|rolling_7d|rolling_30d|media_movel_delta|tendencia_volume|is_feriado|contexto_calendario|target_ola|ocorrencia_ola|origem_dado|arquivo
_origem|data_processamento_utc
4af31913bc0709ca628a69cfc125670a129c6504ffa2fd1f0476b85843cdfc95|18|2|12|4|P2|ALTA|4|0|0|NOITE|10.00|40.00|18.43|45.30|-26.87|BAIXA|0|DIA_UTIL|0|ATENDID
0|PREDICTFY_INCIDENTS_FEATURES_V1|incidentes_predictfy.csv|2026-08-24 00:02:51
b6aeeb8ef5d225e8c65bc55ccd725cb35506ab24a22806d210b8ee1456865ffd|14|2|12|4|P2|ALTA|4|1|0|TARDE|10.00|40.00|18.43|45.30|-26.87|BAIXA|0|DIA_UTIL|0|ATENDID
0|PREDICTFY_INCIDENTS_FEATURES_V1|incidentes_predictfy.csv|2026-08-24 00:02:51
89a1e1bf32a20b8c5c154e647b7425cc25e4a8afa2f8e4b8914adb328c367841|11|2|12|4|P2|ALTA|4|1|0|MANHA|10.00|40.00|18.43|45.30|-26.87|BAIXA|0|DIA_UTIL|0|ATENDID
0|PREDICTFY_INCIDENTS_FEATURES_V1|incidentes_predictfy.csv|2026-08-24 00:02:51
```

Figura 10 — listagem e amostra real do TXT curated.

**Aceite:** 25.588 lidas; zero rejeitadas; 25.588 no MySQL; 25.588 no TXT; COUNT(\*) = COUNT(DISTINCT incident\_feature\_id); segunda execução idempotente.

**Comando executado**

```
az datafactory pipeline-run show ...; az datafactory activity-run query-by-pipeline-run ...
```

**O que faz:** Consulta status, duração e métricas de escrita das duas execuções.

**O que comprova:** Requisito 5 — pipeline Succeeded e 25.588 linhas por destino.

**Comando executado**

```
python3 scripts/collect_evidence.py; jq "{total_registros,chaves_unicas,prioridades_sem_regra}"  
evidencias/azure/13_mysql_validacao.json
```

**O que faz:** Executa as consultas SQL sanitizadas e apresenta contagem, unicidade e cobertura das regras OLA.

**O que comprova:** Requisito 5 — persistência e idempotência no MySQL.

**Comando executado**

```
az storage blob show ... incidentes_predictfy.txt; head -n 4 /tmp/predictfy_curated_sample.txt
```

**O que faz:** Confirma tamanho e data do blob e mostra o cabeçalho e registros reais.

**O que comprova:** Requisito 5 — TXT persistido com colunas derivadas.

## 6. Comentários finais

O desafio materializou um fluxo ETL completo e reproduzível. Os principais aprendizados foram parametrizar linked services com segurança, declarar projeções para o grafo Spark e distinguir funções do ADF das funções SQL. Para melhorar o desempenho do TXT de arquivo único, o upsert foi ajustado para batch de 10.000 e oito partições por hash. O resultado mantém rastreabilidade, separa landing de curated e comprova consistência entre MySQL e TXT.